

DOC NO: ECE495-2025G5 **ISSUE DATE:** 07-04-2026
FILE NAME: G5_assign7_final_report.pdf **VERSION NO:** 1
FILING PATH: <https://github.com/AneekMubarak/MobileToolAssistant/blob/main/docs/>
TITLE: Final Project Report
PROJECT: Mobile Tool Assistant
KEYWORDS: Report, schedule, objectives, constraints, testing

APPROVED BY

Daniel Teng Date
Customer

PREPARED BY:

Aditya Ramakrishnan Date
Design Engineer

Aneek Mohamed Ruksan Date
Design Engineer

Chidubem Emeka-Nwuba Date
Design Engineer

Ethan D'Almeida Date
Design Engineer

REVIEWED BY:

Chandler Janzen Date
Supervisor

REVISION HISTORY

Version No.	Date	Originator	Summary of Changes
1	29-03-2026	E.D	Document created

Table of Contents

Executive Summary	5
Statement of Work	6
Problem Description	6
Problem Background	6
Scope	6
Objectives	6
Constraints	7
Key Considerations	7
Final System Block Diagram	8
System Alternatives and Design Selection	9
Key Design Decisions	9
Design Challenges and Solutions	10
Final Detailed Design	10
Processor (A: 90% C: 10%).....	10
Motor Control (D: 100%)	11
Obstacle Detection (C: 90%, A: 10%).....	12
Power Management (A: 100%)	14
User Tracking (B:100%).....	15
Development Environment.....	22
Testing Setups and Results	22
Actual v/s Expected Project Schedule	23
Recommendations	26
Conclusions	27
References	28

List of Tables

Table 1: Testing Setups and Results	22
Table 2: Expected Project Schedule	23
Table 3: Actual Project Schedule	24
Table 4: Individual Member Contribution Summary	25

List of Figures

Figure 1: Final System Block Diagram	9
Figure 2: Detailed Design - Motor Control Subsystem	12
Figure 3: Detailed Design - Obstacle Detection Subsystem	13
Figure 4: Detailed Design - User Detection: Robot Node	21
Figure 5: Detailed Design - User Detection: Remote Node	21
Figure 6: Gantt Chart for Expected and Actual Project Schedule	24

Executive Summary

This project presents the design and implementation of the Mobile Tool Assistant, a proof-of-concept robotic system developed to reduce the physical strain associated with transporting tools in industrial environments. Existing solutions such as carts and tool belts either require continuous manual effort or place excessive load on workers, creating safety and ergonomic concerns.

To address this problem, our team developed an autonomous, user-following robot capable of carrying tools while navigating indoor environments. The system integrates Ultra-Wideband (UWB) technology for accurate user tracking and Time-of-Flight (ToF) sensors for real-time obstacle detection and avoidance. A central STM32 microcontroller coordinates all subsystems, including motion control, navigation, and power management, enabling reliable and responsive operation.

The prototype successfully demonstrates key functionalities, including maintaining a safe following distance, navigating around obstacles, and supporting remote user control. Testing results show consistent performance in tracking users at walking speeds and avoiding collisions, though payload capacity and motor performance were identified as limiting factors.

Despite challenges such as UWB calibration complexity and system integration delays, the project achieved its primary objective of delivering a functional and cost-effective mobile assistant. The modular design allows for future improvements, including enhanced sensing coverage, more robust hardware integration, and upgraded power systems.

Overall, this work demonstrates the feasibility of an affordable robotic assistant that can improve workplace safety, reduce fatigue, and enhance productivity in demanding environments.

Statement of Work

Problem Description

The project addresses the physical strain of manually transporting tools and equipment in harsh work environments where affordable and accessible transport systems are lacking. The goal is to develop a mobile robotic assistant that can detect a worker, map a route, and follow them to carry tools safely and cost-effectively.

Problem Background

Most workers who work in demanding or harsh environments, i.e. industries or factories, often must carry heavy tools on their person. This becomes an issue especially in industrial environments where operators cannot afford to have excess weight on them, causing safety hazards.

While current methods attempt to address this challenge, they fall short in practice. Workers often rely on push carts, dollies, hand trolleys, and rolling toolboxes to move heavy equipment. These require constant pushing or pulling and can be difficult to maneuver on uneven surfaces or in tight spaces. Other solutions include using a portable tool belt or backpacks. These keep tools within reach but place the full weight directly on the worker's body, increasing fatigue and risk of injury over long shifts.

This forms a need to make a mobile, autonomous, and tool-carrying system that can follow a worker, transport equipment safely, and reduce physical strain without requiring continuous manual operation.

Scope

The project will demonstrate a working **proof-of-concept** of a mobile tool assistant. The team will design, build, and test a small-scale robotic platform that can autonomously follow a single worker and carry a payload of tools across a flat or moderately uneven indoor surface. The work will involve selecting and assembling appropriate hardware such as motors, sensors, a supporting frame, and a battery to create a safe and stable prototype. On the software side, the team will implement core functions such as person-tracking, obstacle avoidance so that the robot can reliably follow a worker in a controlled environment. A user remote will also be developed to allow a worker to toggle between following and standby modes of the robot without requiring constant supervision.

Objectives

The project team shall:

1. Develop and deliver proof of concept of a mobile prototype of a compact and safe robot capable of transporting small tool loads in an indoor industrial environment.
2. Implement a reliable person-following capability that allows the robot to detect and follow a designated worker after activation.
3. Integrate obstacle detection and avoidance to minimize collision risk in a dynamic workspace.

4. Demonstrate usability and safety by producing a working prototype within one academic year and within the approved budget.

Constraints

The following constraints shall limit the design and implementation:

1. **Single-Floor Navigation:** The system shall be restricted to operation on flat, single-level surfaces with little to no debris. The design shall not support stair climbing or traversal of steep inclines.
2. **Indoor Use:** The system shall be designed for operation in indoor environments only. Outdoor operation is excluded due to environmental complexity and weatherproofing requirements.
3. **Payload Capacity:** The system shall be capable of carrying a moderate tool payload only, as limited by motor performance, chassis strength, and available project resources.
4. **Timeframe:** The project shall be executed within the academic year, which constrains the scope to the development of core functionalities rather than advanced features.
5. **Team Size:** The project shall be executed by a team of four members. The limited manpower constrains the project scope, task allocation, and development capacity within the given timeframe.
6. **Budget:** The project shall operate under limited financial resources, restricting the use of high-cost sensors, materials, and hardware.

Key Considerations

The robotic assistant shall operate safely and non-distractingly, maintaining an appropriate distance from workers and functioning predictably to avoid interfering with normal workflows. The design should incorporate reliable load management, robust battery management, and durable, energy-efficient components to meet safety and operational requirements. The project should also emphasize positive social impact by reducing physical strain on workers, ensuring ease of use and accessibility, and reinforcing the role of the robot as an assistant that supports productivity rather than replacing human labor.

Final System Block Diagram

The final system architecture of the Mobile Tool Assistant is designed as a modular, real-time embedded system that integrates user tracking, obstacle detection, motion control, and power management into a cohesive autonomous platform. The system enables the robot to follow a designated user while safely navigating indoor environments.

At a high level, the system consists of six primary subsystems: User Tracking, Navigation (Obstacle Detection), Motion Control, Processing and Communication, Power Management, and User Interface, all coordinated by a central microcontroller.

The User Tracking subsystem utilizes Ultra-Wideband (UWB) technology to estimate the relative position of the user. Two UWB anchors mounted on the robot perform ranging with a remote UWB node carried by the user. Using these distance measurements, the system computes the user's position and heading relative to the robot.

The Navigation subsystem consists of multiple Time-of-Flight (ToF) sensors that continuously monitor the environment for obstacles. These sensors provide distance measurements that are used to detect potential collisions and trigger avoidance maneuvers.

The Motion Control subsystem translates tracking and navigation data into motor commands. It implements control logic to maintain a safe following distance while dynamically adjusting speed and direction. Obstacle avoidance is handled using a state-machine-based approach to ensure predictable and stable behavior.

The Processing and Communication subsystem is implemented on an STM32 microcontroller and acts as the core of the system. It handles sensor data acquisition, UWB communication, decision-making, and coordination between subsystems in real time.

The Power Management subsystem distributes power from a single battery source to all components. It includes voltage regulation for logic-level electronics and direct supply for motors, ensuring stable and efficient operation.

Finally, the User Interface subsystem allows the user to interact with the robot through a remote node and onboard controls. It enables start/stop functionality and provides basic system feedback.

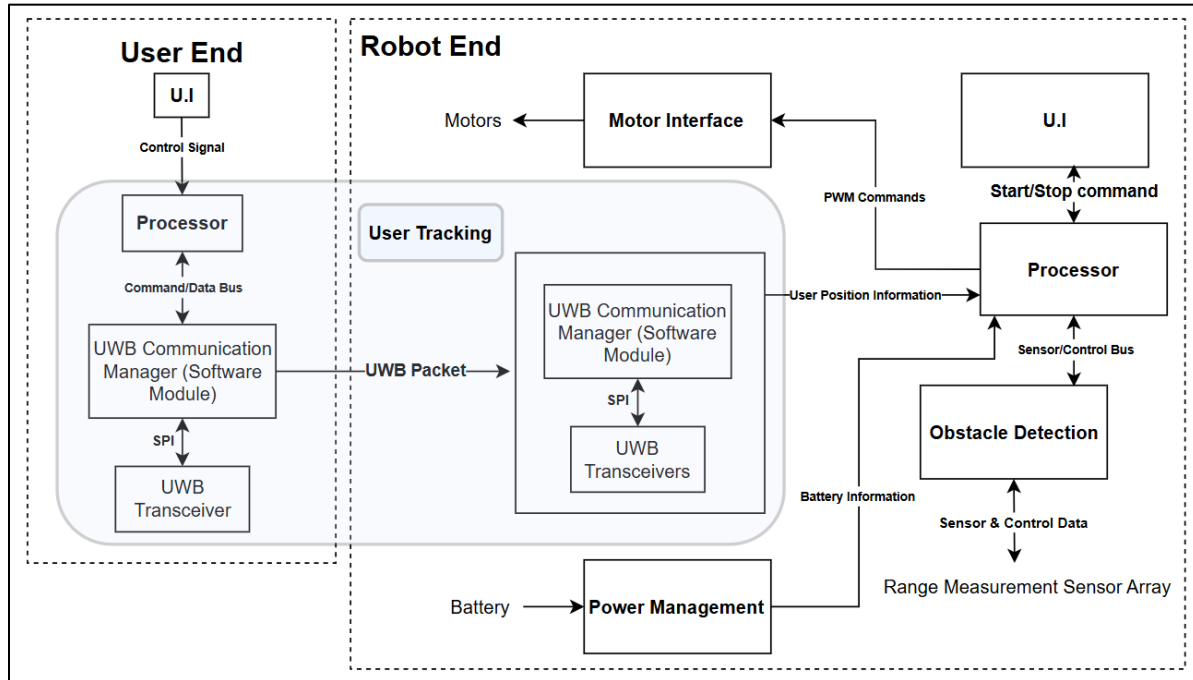


Figure 1: Final System Block Diagram

System Alternatives and Design Selection

Several system architectures were considered during the design phase, including:

- Vision-based tracking with LiDAR navigation
- BLE-based tracking with RSSI and stereo vision
- BLE with IMU-assisted tracking
- UWB-based tracking (final design)

Each alternative was evaluated based on cost, complexity, power consumption, reliability, and implementation feasibility. Vision-based systems offered high accuracy but were computationally expensive and sensitive to lighting conditions. BLE-based approaches were low-cost but suffered from poor positional accuracy due to signal variability.

The UWB-based system was selected as the final design because it provides high-accuracy distance measurements, low latency, and robustness in indoor environments, while remaining within project cost and power constraints. This made it the most balanced and practical solution for a real-time following system.

Key Design Decisions

Several important design decisions were made to ensure system performance and feasibility:

- **Use of dual UWB anchors:** Enabled triangulation of user position rather than relying on a single distance measurement, improving tracking accuracy.
- **RTOS-based control architecture:** Simplified implementation of all subsystems and ensured predictable system behavior.

- **Modular subsystem design:** Allowed independent development and testing of tracking, control, and sensing components.
- **Use of ToF sensors over LiDAR:** Reduced system cost and power consumption while still meeting obstacle detection requirements.
- **Single battery power architecture:** Simplified integration and reduced hardware complexity.

Design Challenges and Solutions

Several challenges were encountered during system development:

- **UWB calibration and instability:**
Achieving accurate distance measurements required careful calibration of antenna delays and filtering of noisy readings. This was addressed through averaging multiple samples and iterative calibration.
- **Integration of multiple subsystems:**
Synchronizing tracking, obstacle detection, and motion control introduced timing and communication challenges. These were mitigated by implementing structured state machines and prioritizing obstacle avoidance over tracking.
- **Limited field of view for tracking:**
The use of two front-mounted UWB anchors restricted tracking to a forward-facing region. This limitation was accepted for the prototype but identified for future improvement.
- **Motor performance under load:**
The robot experienced reduced performance with higher payloads. This was partially mitigated through speed control adjustments and is noted as a limitation of the prototype hardware.

Final Detailed Design

(A – Aditya Ramakrishnan, B – Aneek Mohamed Ruksan, C –Chidubem Emeka-Nwuba, D – Ethan Peter D’Almeida)

Processor (A: 90% C: 10%)

Purpose:

The Processor Module reads all sensor inputs, determines the robot’s motion commands, and sends control signals to the motor drivers. It handles UWB ranging, battery monitoring, state decisions, and safety shutdown actions at the robot’s end.

Inputs:

- UWB Positional Data & Start/Hold Commands
- ToF Sensor Obstacle Detection Distance Data array
- Battery Voltage/Current

Outputs:

- 4× PWM Motor Signals
- UWB Configuration Commands

Parts Required:

- STM32F407DISC0 Development Board
- STM32CubeIDE

Motor Control (D: 100%)

Purpose:

The motion control subsystem is responsible for converting the user's position from the User Tracking subsystem into motor drive signals.

Inputs:

- User Position (x, y) from User Tracking subsystem.
- System State (from obstacle detector and from UI)

Parts Required:

- 2x TT Gear Motor
- 1x L298N Driver

Algorithm:

- Read raw ToF distances from the left, center, and right sensors.
- If in "STANDBY_MODE", immediately stop motors and reset the obstacle state machine.
- Obstacle avoidance is handled by a 3-phase state machine
 - The states are: - OBST_STATE_STOP_BEFORE, OBST_STATE_ACTION, OBST_STATE_STOP_AFTER
- Motor Control Logic:
 - If center < OBSTACLE_THRESHOLD_CLOSE, then the reverse command is given, dodgeAngle is set to 180 degrees, and the state machine is initiated.
 - If center_blocked is true, then the robot will turn towards the side with more clearance, i.e., turn left by 135 degrees and turn right by 45 degrees.
 - If left_blocked then turn right at 45 degrees.
 - If right_blocked then turn left at 135 degrees.
 - If both sides are blocked but the center is clear, move forward at constant speed and maintain a dodgeAngle of 90 degrees.
- UWB Following Logic:
 - If the distance from the remote to the robot is within the range of 0.25 m to 30 m, the angle is within a range of 0 to 180 degrees, and the valid flag is set, then the UWB data will be accepted.
 - The robot will stop if the distance to the remote is less than or equal to 100 cm and will only resume when that distance becomes greater than 115 cm (tolerance measure).
- Speed Control Logic:

- The PWM signal is adjusted by the distance from the remote to the robot, i.e., a smaller distance results in a lower speed, and a higher distance results in a higher speed.

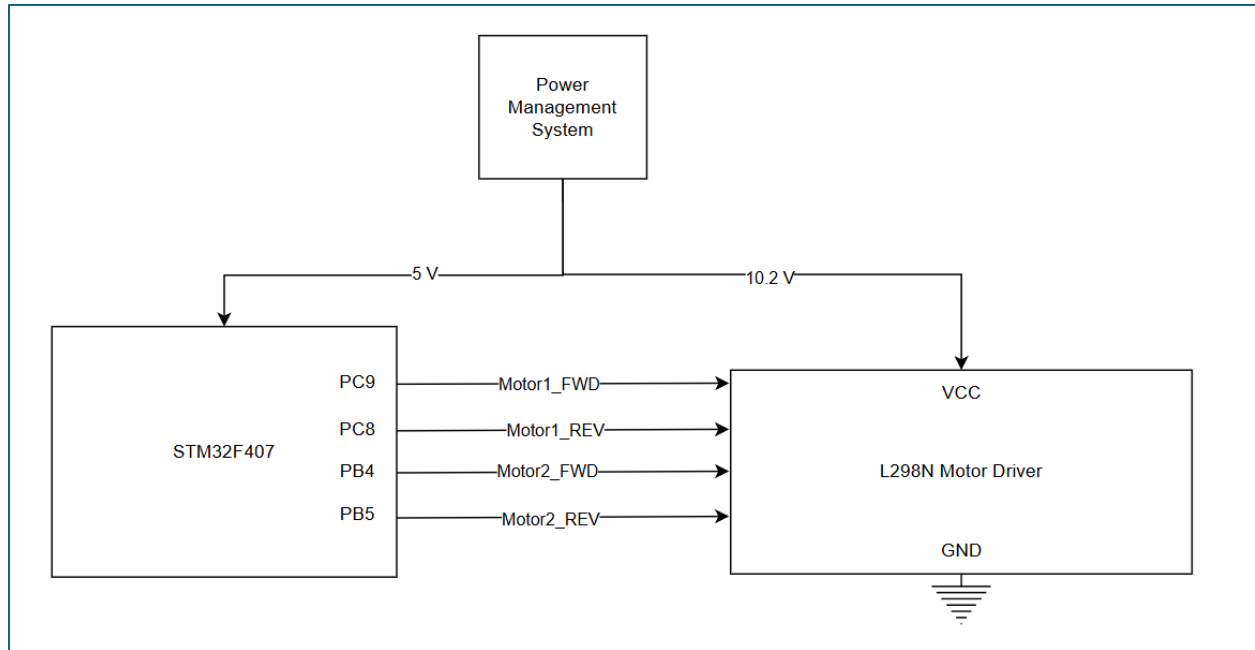


Figure 2: Detailed Design - Motor Control Subsystem

Obstacle Detection (C: 90%, A: 10%)

Purpose:

The purpose of the Obstacle Detection subsystem is to detect obstacles in the robot's path from a minimum distance of 1.5m. This information is then sent to the microcontroller to process and make decisions on whether to keep following the user or avoid an incoming obstacle. The decision is made based on the sensor data stored in a data array for each sensor. The sensors used are 3 VL53L1X placed to provide coverage to the front-facing side of the robot (Refer to Figure 3 for wiring/connection specifications).

Outputs:

Obstacle detection distances from each ToF sensor

Software Algorithm:

- Monitors surroundings using left, center, and right ToF sensors with filtered distance readings
- Determines whether paths are blocked based on distance thresholds (center and side zones)
- Prioritizes obstacle avoidance over normal movement and pauses user-following when needed

- Executes avoidance maneuvers through a state machine to complete multi-step actions reliably
- Reverses immediately when an obstacle is detected very close in front
- Chooses a turn direction (left or right) based on which side has more clearance
- Turns away from obstacles detected on only one side
- Continue moving forward when only side paths are blocked, but the center is clear
- Adjusts its dodge angle to guide the direction of avoidance maneuvers
- Resumes normal user-following behavior when no obstacles are detected

Parts Required:

1. 3 VL53L1X sensors
2. Protoboard to solder buses together
3. Jumper wires (to connect to MCU)

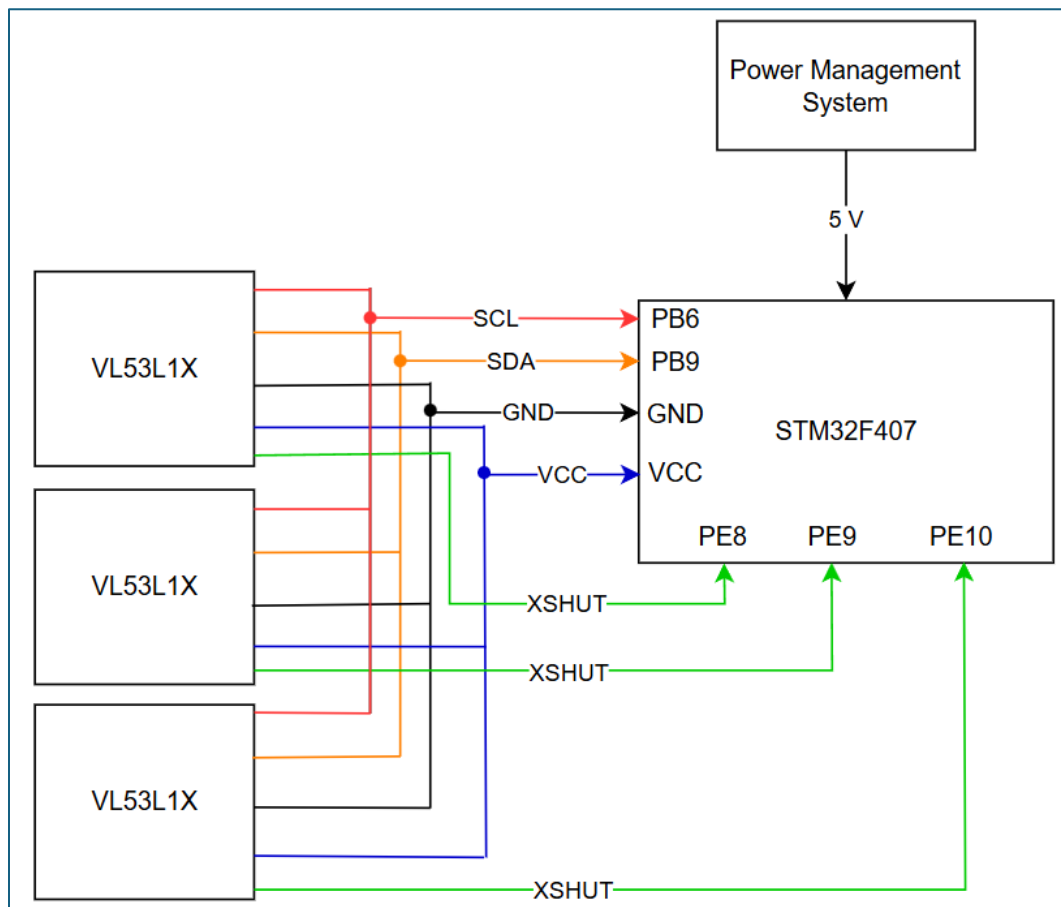


Figure 3: Detailed Design - Obstacle Detection Subsystem

Power Management (A: 100%)

Purpose:

The Power Management Module provides the required voltage for the robot from an 8-cell rechargeable AA battery pack connected in series. The battery pack supplies power directly to the motor driver and feeds a buck converter that steps the voltage down to 5 V for the STM32F407 board. The board then powers and interfaces with UWB modules and distance sensors. This allows the robot to use a single battery source for both motor actuation and control electronics.

Output:

- Approximately 9.6 V nominal battery supply to motor driver
- Approximately 10.2 V when fully charged at startup
- 5 V rail to STM32F407 board through buck converter
- Power to 2× UWB modules through the board
- Power to 3× distance sensors through the board

Parts Required:

- 8 × Energizer rechargeable NiMH AA batteries (1.2 V each)
- 1 × battery holder/connector
- 1 × motor driver
- 1 × buck converter
- 1 × STM32F407 board
- 2 × UWB modules
- 3 × distance sensors
- 2 × yellow DC motors

Power setup:

- Connect the 8 AA rechargeable batteries in series to create the main supply voltage.
- Provide this battery voltage directly to the motor driver to power the motors.
- Feed the same battery voltage into the buck converter.
- Step the battery voltage down to a regulated 5 V output.
- Supply the 5 V output to the STM32F407 board.
- Use the STM32F407 board to power and interface with UWB modules and distance sensors.
- Continue operation while the battery pack provides sufficient voltage.

Power Calculation:

Battery Pack Voltage

$$\text{Nominal Voltage (V)} = 8 * 1.2V = 9.6V$$

Fully charged (measured):

$$V \approx 10.2 V$$

$$\text{Motor Power (2 DC Yellow Motors)} = \sim 10 V / \sim 0.25 A \text{ each (under load)}$$

Rated electrical power per motor:

$$P = 10 V \times 0.25 A = 2.5 W$$

Total motor power:

$$P_{total} = 2 \times 2.5 W = 5.0 W$$

STM32F407 Development Board Power – 5 V / ~0.15 A

$$P = 5 V \times 0.15 A = 0.75 W$$

UWB Modules (2 × DWM1000) – 3.3 V / ~0.16 A each

Total current:

$$I = 2 \times 0.16 A = 0.32 A$$

Total UWB power:

$$P = 3.3 V \times 0.32 A = 1.056 W$$

Distance Sensors (3 × ToF) – 5 V / ~0.02 A each

Total current:

$$I = 3 \times 0.02 A = 0.06 A$$

Total sensor power:

$$P = 5 V \times 0.06 A = 0.30 W$$

Total Average Power:

$$P_{total} = 5.0 W + 0.75 W + 1.056 W + 0.30 W$$

$$P_{total} = 7.106 W$$

User Tracking (B:100%)

Purpose:

The User Tracking subsystem determines the position of the user relative to the front of the robot so that the motion controller can steer toward the user and maintain the desired following distance. The implemented design uses two front-mounted UWB anchors on the robot and one UWB transceiver on the remote carried by the user. Distances from the left and right robot anchors to the remote are measured using Double-Sided Two-Way Ranging (DS-TWR), then converted into user position, distance, and heading angle.

Physical Setup:

The robot node uses two DWM1000 UWB modules mounted at the front of the robot on a rigid horizontal arm. The modules are placed 30 cm apart, centered about the robot midpoint, so that the left anchor is at -15 cm and the right anchor is at +15 cm relative to the robot centerline. This baseline is used directly in the triangulation calculation. The arm is mounted parallel to the front edge of the robot chassis so that both modules face forward and remain fixed relative to one another. The robot-side processing runs on the STM32F407 board as part of the RTOS application. The remote node carried by the user contains one DWM1000 module, one ESP8266MOD, and one pushbutton. The push button is used to transmit a standby control message to the robot. On the robot side, receipt of this message toggles the standby state used by the rest of the system.

Inputs:

- UWB ranging frames received from the remote node
- Standby command embedded in remote UWB communication

Outputs:

- User position relative to robot center, (x, y) , in cm
- User distance from robot center in cm
- User heading angle in degrees
- Standby state flag for higher-level control logic

Parts Required:Robot Node:

1. STM32F407 development board
2. 2 x DWM1000 UWB Modules
3. 1 x rigid front mounting arm for the anchors (30cm)
4. Wiring for SPI, reset, interrupt and power connections

Remote Node:

1. ESP8266MOD Microcontroller
2. 1 x DWM1000 Module
3. 1 x Pushbutton for standby control
4. Portable power source for the remote node (5V).

System Operation:

The robot does not estimate position from a single UWB reading. Instead, it ranges separately to the left and right front anchors and then computes the user's position geometrically. For each anchor, the ranging exchange is performed using DS-TWR. In one ranging cycle, the robot sends a Poll frame, the remote replies with a Response frame, the robot sends a second Poll 2 frame, and the remote replies with a Final frame. Timestamps recorded on both sides are used to compute the signal propagation time while compensating for reply delays. The propagation time is converted to distance, corrected using antenna-delay compensation.

The DS-TWR propagation time is calculated as:

$$t_{prop} = \frac{(rtt_1 \cdot rtt_2) - (reply_1 \cdot reply_2)}{rtt_1 + rtt_2 + reply_1 + reply_2}$$

Where:

- t_{prop} : Signal propagation time between the robot and the remote node
- rtt_1 : Round-trip time measured during the first message exchange (Poll to Response)
- rtt_2 : Round-trip time measured during the second message exchange (Poll2 to Final)
- $reply_1$: Reply delay at the remote node (time between receiving Poll and sending Response)
- $reply_2$: Reply delay at the robot (time between receiving Response and sending Poll 2)

The measured distance (d) is then computed from:

$$d = t_{prop} \cdot c$$

Where c is the speed of light. In the implementation, antenna delay is subtracted before the time is converted to distance.

To reduce noise, the robot averages multiple distance measurements from the left and right anchors before computing the user position. The code uses an averaging count of 5 samples. After valid left and right distances are available, the position is calculated using the fixed baseline between the anchors

Coordinate System:

The coordinate system is defined with the robot center at the origin. Positive x points to the robot's right side, and positive y points forward. With the anchor baseline $B = 30\text{cm}$, the left anchor is located at $(-15,0)$ and the right anchor is located at $(+15,0)$. The user position is calculated from the left and right distances d_L and d_R as:

$$x = \frac{d_L^2 - d_R^2}{2B}$$

$$y = \sqrt{(d_L^2 - \left(x + \frac{B}{2}\right)^2)}$$

The user's distance from the robot center is then:

$$d = \sqrt{x^2 + y^2}$$

The user heading angle is computed from:

$$\theta = \text{atan2}(y, x)$$

In this convention, 0° is directly to the right of the robot, 90° is straight ahead, and 180° is directly to the left.

Standby Control:

The remote node also transmits a control message used to toggle standby mode. On the robot side, if the received message type is identified as a standby frame, the standby flag is toggled. This

allows the user to command the robot to pause or resume tracking using the push button on the remote.

UWB Configuration Parameters:

The DWM1000 modules on both the robot and remote nodes are configured using a fixed set of communication parameters to ensure consistent ranging performance. The following configuration is used for all UWB devices in the system:

- Channel: 5
- Pulse Repetition Frequency (PRF): 16 MHz
- Data Rate: 6.8 Mbps
- Preamble Length: 1024 symbols
- PAC Size: 32
- SFD: Standard
- Antenna Delay: Calibrated per module

Channel 5 and a data rate of 6.8 Mbps were selected to achieve reliable communication with low latency and sufficient range (~30 m) for the intended indoor tracking environment.

All nodes must use identical configuration parameters to ensure successful communication and accurate ranging.

SPI Communication Configuration:

The SPI interface used to communicate with the DWM1000 modules is initialized at a lower clock frequency during startup and configuration, then increased for normal operation.

Initialization Phase SPI Speed: Remote node - 3 MHz; Robot node - 328.125 kHz

Operational Phase SPI Speed: Remote node - 20 MHz; Robot node - 10.5 MHz

UWB Message Structure:

To distinguish between different stages of the DS-TWR protocol and control messages, a custom message format is used for all UWB transmissions.

Each transmitted frame contains a fixed 6-byte payload. The most significant byte (MSB) of the payload represents the message type, while the remaining 5 bytes (lower 40 bits) are used to store timing-related values such as timestamps or reply delays required for DS-TWR calculations.

The following message types are defined:

- 0xA1 – POLL
- 0xB2 – RESPONSE
- 0xA2 – POLL_2
- 0xA3 – FINAL
- 0x69 – STANDBY CONTROL

During operation, the receiving node inspects the message type field to determine the appropriate state transition and processing logic.

In addition to the message type, RESPONSE and FINAL frames include embedded timing

information required for DS-TWR calculations, such as reply delays and timestamps.

This structured message format ensures correct synchronization between the robot and remote nodes and enables reliable execution of the ranging protocol and standby control.

Algorithm:

The User Tracking subsystem is implemented using finite state machines on both the robot node and the remote node. The robot state machine manages the DS-TWR ranging process and distance computation, while the remote state machine handles message responses and control transmission. Each state represents a specific stage in the ranging protocol, ensuring reliable communication, timing control, and error recovery.

Robot State Machine States:

- RSTATE_SEND_POLL
- RSTATE_WAIT_RESPONSE_1
- RSTATE_SEND_POLL_2
- RSTATE_WAIT_RESPONSE_2
- RSTATE_COMPUTE
- RSTATE_ERROR_RECOVERY

Remote State Machine States:

- REMOTE_WAIT_M1
- REMOTE_SEND_M2
- REMOTE_WAIT_TX_DONE
- REMOTE_WAIT_M3
- REMOTE_SEND_M4
- REMOTE_WAIT_TX_DONE_FINAL

The following algorithm describes the real-time execution of the state machines on both robot and remote nodes.

1. Initialize SPI, reset, and interrupt interfaces and apply the predefined UWB configuration parameters.
2. Set the robot ranging state to RSTATE_SEND_POLL for the active anchor and set the remote state to REMOTE_WAIT_M1.
3. For the active robot anchor, transmit a POLL frame to the remote node.
4. On the remote node, wait in REMOTE_WAIT_M1 until a valid POLL frame is received. If the received frame is invalid or a timeout occurs, clear status flags, reset the remote session, and return to REMOTE_WAIT_M1.
5. After receiving a valid POLL frame, the remote records the reception timestamp, computes the reply delay, and enters REMOTE_SEND_M2. It then transmits a delayed RESPONSE frame containing the reply timing information and moves to REMOTE_WAIT_TX_DONE.
6. The robot waits in RSTATE_WAIT_RESPONSE_1 for the Response frame. If the frame is invalid or a timeout occurs, the robot enters RSTATE_ERROR_RECOVERY, clears the transceiver state, and restarts the ranging sequence for that anchor.
7. After a valid RESPONSE frame is received, the robot stores the received timing data, transmits a second POLL_2 frame, and enters RSTATE_WAIT_RESPONSE_2.

8. On the remote node, once transmission of the RESPONSE frame is complete, the state machine moves to REMOTE_WAIT_M3. The remote waits for a valid POLL_2 frame. If reception fails or the wrong message type is received, the remote resets its session and returns to REMOTE_WAIT_M1.
9. After receiving POLL_2, the remote records the second reception timestamp, computes the second delayed reply, enters REMOTE_SEND_M4, and transmits the FINAL frame. It then waits in REMOTE_WAIT_TX_DONE_FINAL and, after successful transmission, returns to REMOTE_WAIT_M1.
10. The robot receives the FINAL frame in RSTATE_WAIT_RESPONSE_2. Using the stored timestamps and reply values, it computes the DS-TWR propagation time, applies antenna-delay compensation, and converts the result to distance for the active anchor. It then enters RSTATE_COMPUTE.
11. If the computed distance is valid, store it in the averaging buffer for that anchor. If the measurement is invalid, discard it and restart the ranging sequence.
12. Repeat the ranging process for both left and right anchors to obtain one valid distance measurement from each.
13. Continue collecting measurements until the required number of valid samples is reached for both anchors. Average the left and right distances using the configured sample count of 5 to reduce noise before position estimation.
14. Use the averaged left and right distances to compute the user position (x, y) , total distance from the robot center, and heading angle relative to the front of the robot.
15. Check the received message type for a standby control message. If a standby frame is detected, toggle the standby flag so the robot pauses.
16. Output the updated user position, distance, angle, and standby state to be used by the motion control subsystem.
17. Repeat the full process continuously while the system is powered.

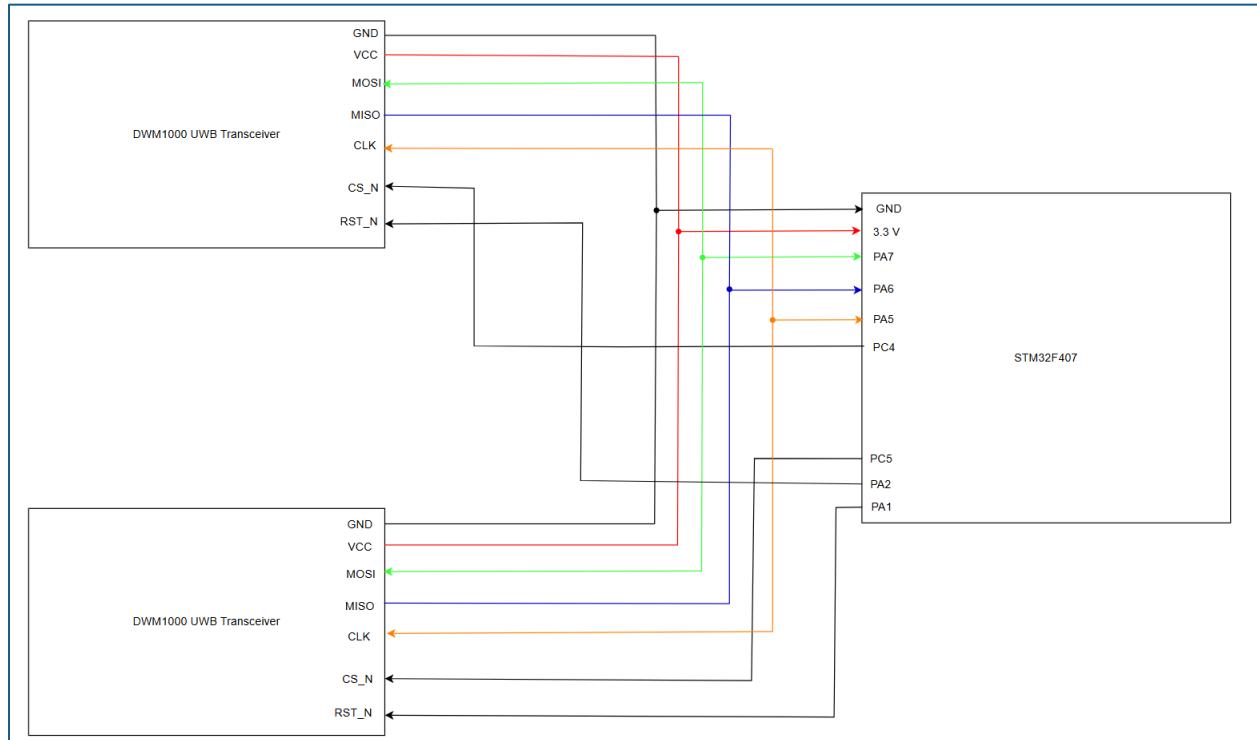


Figure 4: Detailed Design - User Detection: Robot Node

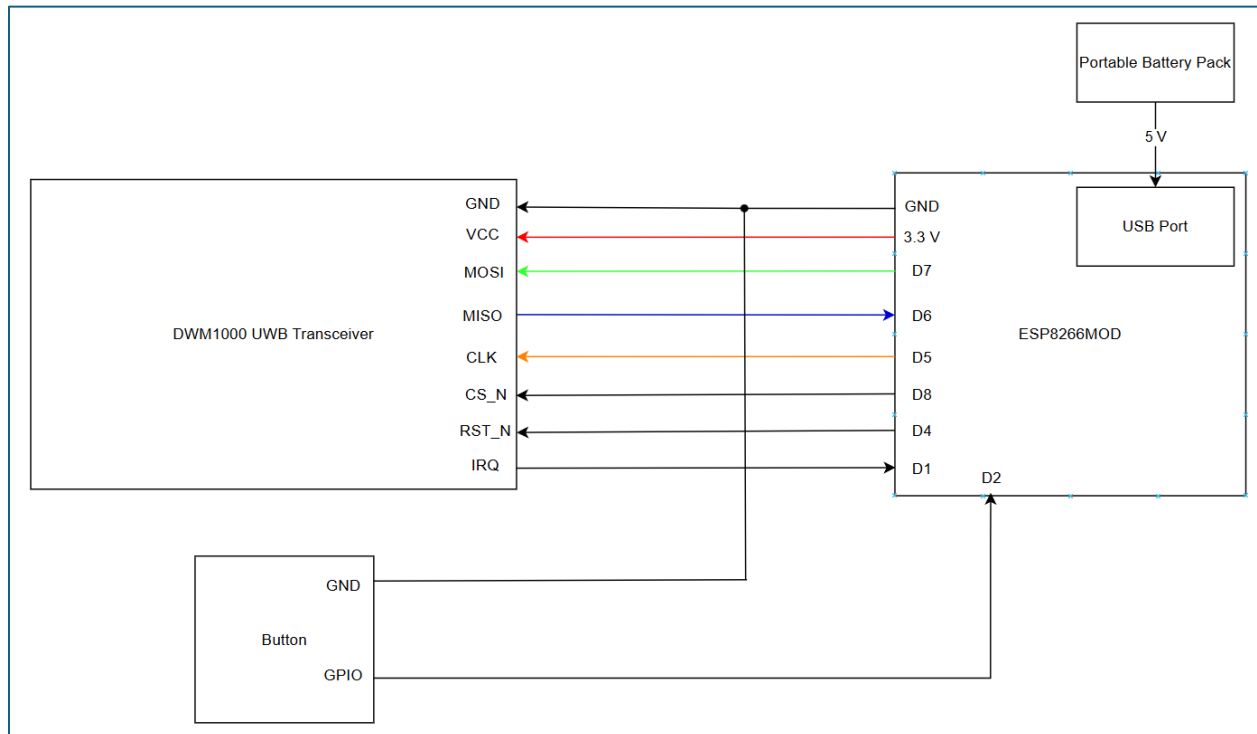


Figure 5: Detailed Design - User Detection: Remote Node

Development Environment

1. **Hardware Used:** -
 - Multimeter
 - Soldering Iron
 - BambuLab P1S 3D printer
2. **Software Used:** -
 - STM32Cube IDE v1.19.0
 - Serial Debug Assistant
 - Arduino IDE v2.3.7

Testing Setups and Results

Table 1: Testing Setups and Results

Test ID	Requirement	Test Objective	Test Procedure	Expected Result	Actual Result
T1	Follow distance: $1\text{m} \pm 0.5$	Verify the robot maintains a proper following distance from the remote.	Place the robot at varying distances (0.5m to 2m) and measure the actual following distance	The robot should be able to maintain an average distance of $1\text{m}, \pm 0.5\text{m}$ in all trials.	Robot maintains an average following distance of $1 \pm 0.5\text{m}$ in all the trials we conducted.
T2	Follow speed: $5\text{ km/h}, \pm 1\text{ km/h}$	Verify the robot's ability to follow a moving user at a specified speed.	The user should move at speeds up to 5 km/h to measure the robot's following feature.	The robot should be able to track and follow a user in the environment it is operating in.	The robot was successfully able to track and follow the user at walking speeds of up to 6 km/h .
T3	Payload: 5 kg capacity	Verify the robot's ability to carry a payload safely.	Place a payload on the robot that weighs up to 5 kg and checks if it can follow the user in its operating environment.	The robot should be able to track and follow a user with a payload of up to 5kg in weight.	The robot was able to track and follow a user with a payload of up to 1kg without any issues. An increase in weight resulted in significant speed drops due to motor constraints.

T4	Obstacle Detection	Verify the robot's ability to detect and avoid obstacles.	Place static obstacles within 50 cm of the robot's path.	The robot should be able to detect obstacles within 50 cm of it and adjust its path to continue tracking and following the user.	The robot was able to successfully detect obstacles within 50 cm of its path and adjust its course to continue following the user.
----	--------------------	---	--	--	--

Actual v/s Expected Project Schedule

Table 2: Expected Project Schedule

Task ID	Task Name	Estimated Hours	Responsible Member	Precedence	Expected Deadline
1	Detailed Design (Assignment)	20h	Chidubem	None	27 th Nov, 2025.
2	Parts Selection and Ordering	10h	Aditya	T1	1 st Dec, 2025.
3	UWB Tracking Module (User Tracking)	40h	Aneek	T2	21 st Jan, 2026.
4	Obstacle Detection	45h	Chidubem	T2	21 st Jan, 2026.
5	Motion Control Algorithm (Motor wiring)	21h	Ethan	T3, T4	30 th Jan, 2026
6	User Interface (and User Node Assembly)	20h	Aneek	T3	30 th Jan, 2026
7	Power Management System	24h	Aditya	T1	21 st Jan, 2026
8	System Assembly and Integration (Constructing Prototype)	30h	Chidubem	T5, T6, T7	15 th Feb, 2026
9	Chassis Design and Printing/Ordering	15h	Aditya	T8	15 th Feb, 2026
10	System Testing and Debugging	37h	Ethan	T9	28 th Feb, 2026
11	System Refining and Performance Improvement	16h	Aneek	T10	28 th Feb, 2026

Table 3: Actual Project Schedule

Task ID	Task Name	Actual Hours Spent	Responsible Member	Precedence	Completion Date
1	Detailed Design (Assignment)	20h	Chidubem	None	27 th Nov 2025.
2	Parts Selection and Ordering	25h	Aditya	T1	27 th Jan 2026.
3	UWB Tracking Module (User Tracking)	175h	Aneek	T2	12 th Mar 2026.
4	Obstacle Detection	45h	Chidubem	T2	4 th Mar 2026.
5	Motion Control Algorithm (Motor wiring)	43h	Ethan	T3, T4	24 th Mar 2026
6	User Interface and User Node Assembly	5h	Aneek	T3	13 th Mar 2026
7	Power Management System	24h	Aditya	T1	20 th Mar 2026
8	System Assembly and Integration (Constructing Prototype)	30h	Chidubem	T5, T6, T7	23 rd Mar 2026
9	Chassis Design and Printing/Ordering	15h	Aditya	T8	19 th Mar 2026
10	System Testing and Debugging	40h	Ethan	T9	24 th Mar 2026
11	System Refining and Performance Improvement	37h	Aneek	T10	24 th Mar 2026

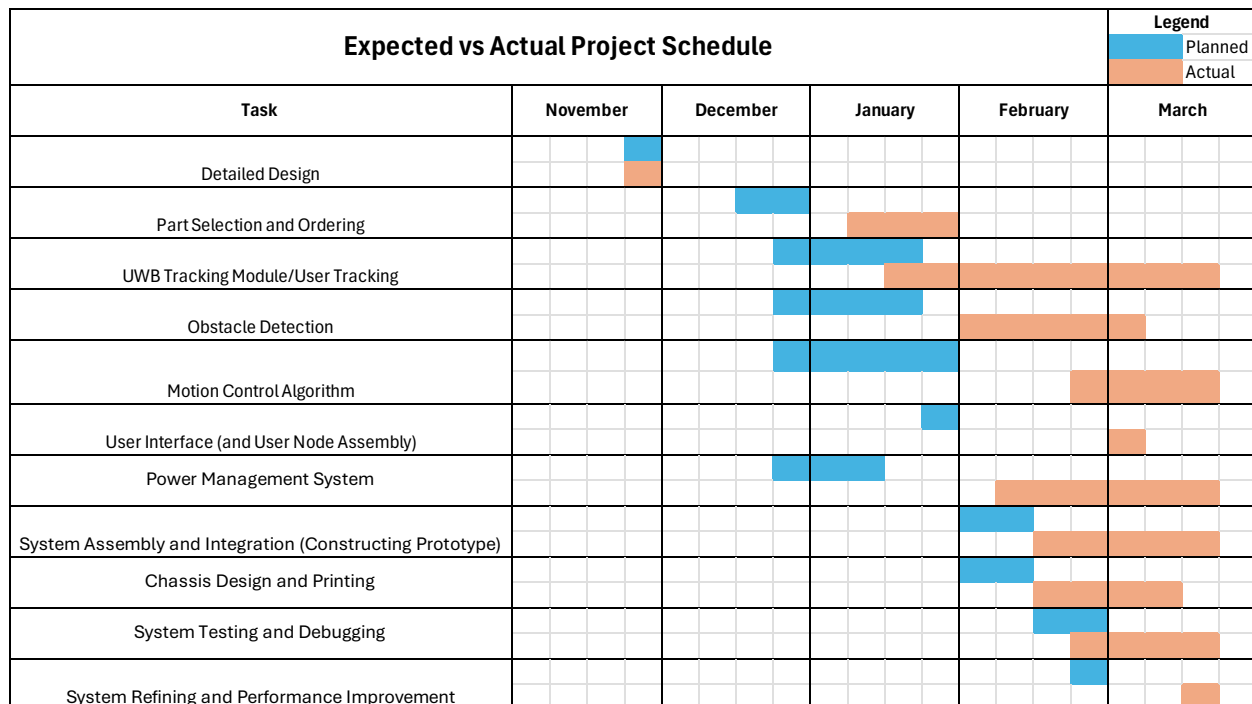


Figure 6: Gantt Chart for Expected and Actual Project Schedule

Table 4: Individual Member Contribution Summary

Member	Total Hours	Key Contributions
Aneek Mohamed Ruksan	180	UWB User Tracking, UI/User Node
Aditya Ramakrishnan	120	Motor Control Algorithm, Power Management System, Obstacle Avoidance, Chassis Design and Printing, System Refining and Improvement, System Assembly
Ethan D’Almeida	70	UWB calibration support and testing, documentation, obstacle detection, and avoidance improvements
Chidubem Emeka-Nwuba	75	Obstacle Detection Subsystem Assembly and Integration, Software architecture: implementation, management, and subsystem task integration

Project Analysis and Reflection

The project did not follow the original schedule mainly because some tasks were much more difficult than expected. The biggest issue was our lack of experience with UWB technology, which led to large delays early on the project and later affected many tasks. The largest difference was in the UWB tracking module (Task-3), which took us 175 hrs to complete and get fully working rather than the planned 40 hrs. Main challenges in this task that can be attributed to the delay include: -

- Calibrating the antenna delay, which required multiple rounds of testing and adjustments.
- Implementing the double-sided two-way ranging algorithm correctly.
- Fixing drift in tracking and unstable position and direction readings.

This task was required because it was the highlight feature of our project and had to be completed before others could start, which caused a chain reaction that pushed back multiple tasks, such as motion control and system integration.

Other tasks also took longer than expected, such as: -

- Parts Selection (Task-2): Took 25 hrs instead of 10 hrs due to re-designs and part availability.
- Motion Control (Task-5): Took 43 hrs instead of 21 hrs due to issues in integration.
- System Refinement (Task-11): Took 37 hrs instead of 16 hrs due to extra testing and improvements, specifically in the obstacle avoidance algorithm.

However, some time was saved in the User Interface (Task-6), which only took 5 hrs instead of 20 hrs because the design was simplified.

Mitigation Strategies

To deal with the delays, our team made some adjustments: -

- Ethan helped Aneek with UWB calibration and testing, which improved progress on one of the more difficult tasks in the project.
- Some tasks, such as obstacle detection and the power management system, were done in parallel to reduce any further delays.
- Features like the user interface were greatly simplified to save time.
- Instead of waiting till the end, we tested parts of the system to catch any problems earlier.
- More time was spent on testing and refinement to ensure the system worked reliably.

Recommendations

While the final system meets the core project requirements, it was developed as a proof-of-concept prototype with several design trade-offs that impact performance, reliability, and scalability. One key limitation was in the UWB tracking system, where the use of only two sensors provided a 180° field of view, constraining the user to remain in front of the robot; in the intended design, adding a third sensor at the rear would enable full 360° coverage and allow continuous tracking regardless of user position. Similarly, the prototype relied primarily on ToF sensors for obstacle detection, which limited environmental awareness; integrating a 360° LIDAR sensor in the final design would provide comprehensive planar coverage for navigation, while the existing ToF sensors could be retained for vertical distance measurement, resulting in improved obstacle detection and avoidance across a wider range of scenarios. Additionally, the use of jumper wire connections in the prototype, while beneficial for rapid development, introduced reliability issues such as loose connections, signal instability, and inconsistent test results; transitioning to custom PCBs in the intended design would significantly enhance electrical reliability, mechanical robustness, and signal integrity, while also simplifying assembly, reducing noise susceptibility, and enabling more accurate and repeatable system testing. Finally, future iterations will focus on improving the power system by transitioning to an industry-standard lithium-ion battery module, like those used in robotic vacuum platforms such as Roomba; this would involve a sealed, rechargeable battery pack with an integrated battery management system, enabling safer operation, higher energy density, and more reliable long-term performance.

Conclusions

Our project successfully developed and tested a mobile robotic assistant, demonstrating a cost-effective solution for transporting heavy tools in demanding industrial environments. Our design integrates Ultra-Wideband (UWB) and Time-of-Flight (ToF) sensor technologies, providing a robust platform capable of autonomously tracking a worker while maintaining safe distances and navigating around dynamic obstacles. Furthermore, our implementation of a remote-based standby control system ensures the robot operates as an intuitive assistant that responds to user commands without requiring constant manual supervision.

The design relies on a modular architecture, utilizing a central STM32 microcontroller to process UWB ranging data and ToF obstacle inputs, which are then translated into precise motor commands. The UWB modules allow for reliable user localization via trilateration, while the ToF sensor array provides real-time environmental awareness, enabling the robot to navigate safely in tight, indoor workspaces. These components allowed our team to implement a functional proof-of-concept using a budget-conscious approach that prioritizes both safety and ease of use.

While further refinement is necessary, specifically regarding long-term signal stability and power system optimization, the prototype successfully proves the core concepts of autonomous person-following and obstacle avoidance. The development process has provided a clear roadmap for future iterations, including moving to custom PCB designs and higher-density battery solutions. Ultimately, this work contributes to the development of affordable, high-utility robotic assistants designed to reduce physical strain and increase productivity for workers in industrial and construction settings.

References

Decawave Ltd. (2017). DW1000 product datasheet (Version 2.22) [Data sheet].
<https://www.qorvo.com/products/p/DW1000>

Decawave Ltd. (2017). DW1000 user manual (Version 2.18) [User manual].
<https://www.qorvo.com/products/p/DW1000>

Qorvo. (n.d.). DWM1000 ultra-wideband module (Version 2.0) [Data sheet].
<https://www.qorvo.com/products/d/da007948>

Amazon Web Services, Inc. (2017). FreeRTOS reference manual (Version 10.0.0) [Reference manual].
https://www.freertos.org/media/2018/FreeRTOS_Reference_Manual_V10.0.0.pdf

STMicroelectronics. (2024). VL53L1X time-of-flight ranging sensor (Rev. 8) [Data sheet].
<https://www.st.com/resource/en/datasheet/vl53l1x.pdf>

STMicroelectronics. (2026). STM32F407xx Development Board (Rev. 11) [Data sheet].
<https://www.st.com/resource/en/datasheet/dm00037051.pdf>